

Manual de Instruções para a Interface e Relatório

Autor(es): Gabriel Sanches da Silva

E-mail(s): gabrielss2403@usp.br

Resumo

Este documento é dividido em duas etapas principais: a primeira dela é um manual de instruções de como usar a interface, definindo arquitetura de arquivos e como interpretar resultados; a segunda é voltada para pesquisadores e/ou desenvolvedores que precisam enxergar a parte mais técnica da interface, dos códigos utilizados, e do modelo de otimização elaborado.

Sumário

1	Introdução	5
1.1	Visão Geral	5
1.2	Objetivo	5
1.3	Tecnologias Utilizadas	5
2	Arquitetura do Sistema	5
2.1	Estrutura de Diretórios	5
2.2	Fluxo de Dados	5
3	Módulos Principais	6
3.1	Padronização de Dados	6
3.1.1	Função: <code>padroniza_dataframe()</code>	6
3.2	Construção de Bases de Dados	6
3.2.1	Função: <code>planilha_dep()</code>	6
3.2.2	Função: <code>concat_df()</code>	7
3.3	Base de Dados Completa	7
3.3.1	Função: <code>base_dados()</code>	7
3.3.2	Função: <code>base_pior_caso()</code>	7
4	Execução do Modelo	7
4.1	Função: <code>execute()</code>	7
4.2	Função: <code>Novo_edit_config()</code>	8
4.3	Função: <code>roda_script()</code>	8
5	Geração de Relatórios	8
5.1	Análise de Espaços Livres	8
5.1.1	Função: <code>criar_visualizacao_de_vazias()</code>	8
5.2	Visualizações	9
5.2.1	Função: <code>visualizacao_completa()</code>	9
5.2.2	Função: <code>visualizacao_curso()</code>	9
5.2.3	Função: <code>visualizacao_dep()</code>	10
5.3	Padronização de Horários	10
5.3.1	Função: <code>padronizar_horario_intranet()</code>	10
5.4	Planilhas para Intranet	10
5.4.1	Função: <code>planilhas_sti()</code>	10
6	Preenchimento de Planilhas	11
6.1	Função: <code>preencher_planilha_dados()</code>	11
6.2	Função: <code>escolhas_preenchimento()</code>	11
6.3	Função: <code>preenchimento()</code>	12
7	Classe de Apoio	12
7.1	Classe: <code>Tooltip</code>	12

8	Tratamento de Erros	13
8.1	Tipos de Erros Tratados	13
8.1.1	PermissionError	13
8.1.2	KeyError	13
8.1.3	subprocess.CalledProcessError	13
9	Interface Gráfica	13
9.1	Janela Principal	13
9.2	Menu de Relatórios	14
10	Fluxo de Trabalho Típico	14
10.1	Preparação de Dados	14
10.2	Construção das Bases	14
10.3	Execução do Modelo	15
10.4	Geração de Relatórios	15
10.5	Preenchimento de Planilhas	16
11	Boas Práticas	16
11.1	Preparação de Arquivos	16
11.2	Nomenclatura	16
11.3	Organização	16
12	Solução de Problemas	17
12.1	Problemas Comuns	17
12.1.1	Erro: Turmas não identificadas	17
12.1.2	Erro: Horário não padronizado	17
12.1.3	Erro: Arquivo está aberto	17
12.1.4	Erro: Coluna não encontrada	17
12.2	Logs e Depuração	17
13	Apêndices	17
13.1	Apêndice A: Formato de Planilhas de Entrada	17
13.1.1	Planilha de Departamento	17
13.1.2	Planilha de Salas	18
13.2	Apêndice B: Saídas do Sistema	18
13.2.1	Dados da Solução do Modelo	18
13.2.2	Visualização Completa	18
13.2.3	Planilha de Espaços Livres	19
13.2.4	Planilhas da Intranet	19
13.3	Apêndice C: Estrutura do Código	19
13.3.1	Imports	19
13.3.2	Funções por Categoria	19
13.4	Apêndice D: Algoritmos Principais	21
13.4.1	Detecção de Intervalos Livres	21
13.4.2	Padronização de Horários	21
13.4.3	Preenchimento de Planilhas	22
13.5	Apêndice E: Mensagens de Erro e Soluções	23
13.6	Apêndice F: Configurações Avançadas	23
13.6.1	Modificação de Intervalos de Horário	23

13.6.2	Adição de Novos Departamentos	24
13.6.3	Adição de Novos Cursos	24
13.6.4	Personalização de Cores	24
13.7	Apêndice G: Glossário	25
14	Referências	25
14.1	Bibliotecas Python	25
14.2	Documentação USP	26
15	Controle de Versões	26
15.1	Histórico de Versões	26
15.2	Desenvolvedor	26
16	Considerações Finais	26
16.1	Limitações Conhecidas	26
16.2	Desenvolvimentos Futuros	26
16.3	Contato e Suporte	27
16.4	Licença	27

1 Introdução

1.1 Visão Geral

Este documento descreve o sistema de alocação de salas desenvolvido para o Instituto de Ciências Matemáticas e de Computação (ICMC) da USP. O sistema é uma interface gráfica desenvolvida em Python usando Tkinter que automatiza o processo de alocação de disciplinas em salas de aula.

1.2 Objetivo

O sistema tem como objetivo principal facilitar a gestão de espaços físicos do ICMC, permitindo:

- Consolidação de dados de múltiplas fontes
- Execução de modelos de otimização para alocação
- Geração de relatórios e visualizações
- Preenchimento automático de planilhas administrativas

1.3 Tecnologias Utilizadas

- **Python 3.x:** Linguagem de programação principal
- **Tkinter:** Interface gráfica
- **Pandas:** Manipulação de dados
- **Openpyxl:** Manipulação de arquivos Excel
- **Subprocess:** Execução de scripts externos

2 Arquitetura do Sistema

2.1 Estrutura de Diretórios

O sistema utiliza a seguinte estrutura de diretórios:

2.2 Fluxo de Dados

1. **Entrada:** Planilhas dos departamentos (SME, SMA, SCC, SSC) e dados do JupyterWeb
2. **Processamento:** Padronização, validação e consolidação
3. **Modelagem:** Execução do modelo de otimização
4. **Saída:** Relatórios, visualizações e planilhas preenchidas

3 Módulos Principais

3.1 Padronização de Dados

3.1.1 Função: `padroniza_dataframe()`

Propósito: Padroniza dataframes para formato unificado.

Parâmetros:

- `file_name`: Nome do arquivo a ser padronizado
- `header_row`: Linha onde está o cabeçalho
- `ano`: Ano dos dados

Funcionalidades:

1. Remove quebras de linha dos cabeçalhos
2. Padroniza nome da coluna "Sala"
3. Filtra disciplinas que devem ser alocadas no ICMC
4. Adiciona colunas de horários faltantes (Horário 3 e 4)
5. Valida e corrige traços em horários
6. Adiciona colunas para proibição de alocação

Validações:

- Verifica turmas não identificadas
- Valida formato de horários (padrão: "Segunda - 08:10/09:50")
- Detecta múltiplos traços ou quebras de linha

3.2 Construção de Bases de Dados

3.2.1 Função: `planilha_dep()`

Propósito: Interface para construção de bases de dados.

Tipos de Bases:

1. **Base de Dados das Aulas** (`jupiter=False`):
 - Consolida planilhas de todos os departamentos
 - Inclui dados das salas
 - Adiciona ano dos dados
2. **Base de Dados do JúpiterWeb** (`jupiter=True`):
 - Consolida dados de inscrições
 - Agrupa por código de disciplina
 - Permite múltiplos arquivos de outros institutos

3.2.2 Função: `concat_df()`

Propósito: Concatena múltiplos arquivos em uma base única.

Processo:

1. Lê cada arquivo individualmente
2. Identifica linha do cabeçalho
3. Padroniza formato
4. Cria arquivo Excel com múltiplas sheets

3.3 Base de Dados Completa

3.3.1 Função: `base_dados()`

Propósito: Cria base de dados final para o modelo.

Modos de Operação:

1. **Modo Normal** (`pior_caso=False`):
 - Cruza dados das aulas com JúpiterWeb
 - Adiciona dados de ingressantes
 - Inclui informações de espelho
 - Executa script `jupiter sheet maker.py`
2. **Modo Pior Caso** (`pior_caso=True`):
 - Compara duas bases de anos diferentes
 - Mantém maior número de inscritos
 - Útil para planejamento conservador

3.3.2 Função: `base_pior_caso()`

Algoritmo:

```
1 Para cada disciplina em df1 (ano mais recente):  
2     Se disciplina existe em df2 (ano anterior):  
3         Se inscritos(df2) > inscritos(df1):  
4             Atualizar df1 com dados de df2
```

4 Execução do Modelo

4.1 Função: `execute()`

Propósito: Interface principal para verificação e execução.

Opções:

1. Verificação de Dados: Valida consistência antes da execução
2. Execução do Modelo: Configura e executa otimização

4.2 Função: Novo_edit_config()

Propósito: Configuração de parâmetros do modelo.

Parâmetros Configuráveis:

Parâmetro	Recomendado	Descrição
Peso de alocação (w_x)	1	Importância de alocar disciplinas
Peso de troca (w_y)	500	Penalidade por trocar de sala
Peso de agrupamento (w_v)	1	Importância de agrupar cursos
Peso de superlotação (w_z)	10	Penalidade por salas cheias
Índice de superlotação (α)	0.85	Limiar de ocupação
Peso de salas vazias	500	Preferência por manter salas livres

Tabela 1: Parâmetros do Modelo de Otimização

4.3 Função: roda_script()

Propósito: Executa scripts Python externos.

Características:

- Captura saída em tempo real
- Exibe em janela scrollable
- Trata erros de execução
- Usa `sys.executable` para compatibilidade

5 Geração de Relatórios

5.1 Análise de Espaços Livres

5.1.1 Função: criar_visualizacao_de_vazias()

Propósito: Identifica horários disponíveis nas salas.

Processo:

1. Carrega visualização completa da solução
2. Define intervalo de horários (07:00 - 23:30, a cada 30min)
3. Para cada sala:
 - Identifica células sem alocação
 - Registra intervalos contínuos disponíveis
 - Marca com preenchimento amarelo
4. Identifica salas completamente desocupadas
5. Gera dois arquivos:

- Visualização gráfica (Excel)
- Planilha de dados (Excel)

Algoritmo de Detecção:

```
1  Para cada linha (dia da semana):  
2      inicio_vazio = None  
3  
4      Para cada coluna (horário):  
5          Se celula esta vazia:  
6              Se inicio_vazio eh None:  
7                  inicio_vazio = coluna_atual  
8          Senao (celula ocupada):  
9              Se inicio_vazio nao eh None:  
10                 Registrar intervalo (inicio_vazio, coluna_atual  
11                 )  
12                 inicio_vazio = None
```

5.2 Visualizações

5.2.1 Função: visualizacao_completa()

Propósito: Gera grade horária completa de todas as salas.

Estrutura:

- Uma tabela por sala
- Linhas: dias da semana (Segunda a Sábado)
- Colunas: horários a cada 30 minutos
- Células mescladas para aulas de múltiplos horários
- Código de cores: verde para aulas alocadas

5.2.2 Função: visualizacao_curso()

Propósito: Separa visualização por curso.

Cursos Contemplados:

- BMAcc (Bacharelado em Matemática Aplicada e Computação Científica)
- BMA (Bacharelado em Matemática Aplicada)
- LMA (Licenciatura em Matemática)
- MAT-NG (Matemática - Noturno Geral)
- BECD (Bacharelado em Estatística e Ciência de Dados)
- BCC (Bacharelado em Ciência da Computação)
- BSI (Bacharelado em Sistemas de Informação)
- BCDados (Bacharelado em Ciência de Dados)

5.2.3 Função: `visualizacao_dep()`

Propósito: Separa visualização por departamento.

Departamentos:

- SME (Matemática Aplicada e Estatística)
- SMA (Matemática)
- SCC (Ciências de Computação)
- SSC (Sistemas de Computação)
- Outras (disciplinas de outros institutos)

5.3 Padronização de Horários

5.3.1 Função: `padronizar_horario_intranet()`

Propósito: Converte horários para formato padrão da intranet.

Regras de Conversão:

- Remove espaços e converte para minúsculas
- Horários antes das 18h: minutos arredondados para :00
- Horários de fim $\leq 18h$:
 - Se minutos < 30 : hora anterior :30
 - Se minutos > 30 : mesma hora :30
- Horários de fim $> 18h$:
 - Se $0 < \text{minutos} < 30$: :00
 - Se minutos > 30 : :30
 - Se minutos $= 0$: hora anterior :30

Exemplo:

```
1 Entrada:  "Segunda - 08:10/09:50 "  
2 Saída:    "Segunda - 08:00/09:30 "
```

5.4 Planilhas para Intranet

5.4.1 Função: `planilhas_sti()`

Propósito: Gera planilhas no formato da intranet do ICMC.

Saídas:

1. **Planilha Completa:** Todas as alocações em um arquivo
2. **Planilhas por Sala:** Um arquivo CSV para cada sala

Colunas Geradas:

- Sala
- Disciplina (código)
- Descrição (nome completo)
- Aula (sempre "Aula")
- Dia e hora início
- Dia e hora fim
- Docente (NUSP)
- Compartilhada com outra sala? (S/N)

Tratamento de Laboratórios:

- 6-303/6-304 → 6-303 (compartilhada: S)
- 6-305/6-306 → 6-305 (compartilhada: S)

6 Preenchimento de Planilhas

6.1 Função: preencher_planilha_dados()

Propósito: Interface para preenchimento automático.

Modos:

1. **Completo:** Preenche todas as alocações da solução
2. **Parcial:** Permite selecionar quais aulas fixar

6.2 Função: escolhas_preenchimento()

Propósito: Interface para seleção de aulas a fixar.

Funcionamento:

1. Exibe tabela scrollable com todas as aulas
2. Colunas: Disciplina, Horário, Sala, Inscritos, Checkbox
3. Usuário marca aulas desejadas
4. Gera arquivo "...com *Escolhas.xlsx*"

6.3 Função: preenchimento()

Propósito: Executa preenchimento das planilhas.

Processo:

1. Para cada arquivo de elenco:
 - Identifica linha do cabeçalho
 - Para cada disciplina:
 - Verifica se está na solução
 - Limpa coluna de salas
 - Cruza horários com solução
 - Preenche salas correspondentes
 - Salva "...Preenchido.xlsx"
2. Para base de dados (se fornecida):
 - Processa todas as sheets
 - Atualiza coluna "Sala" com formato: "sala1, sala2, ..."
 - Salva "...Preenchido.xlsx" ou "...Parcial.xlsx"

Formato de Salas:

```
1 Disciplina com 3 horários:
2 Sala: "3-009, 3-010, 3-011"
3 (ordem corresponde a Horário 1, 2, 3)
```

7 Classe de Apoio

7.1 Classe: Tooltip

Propósito: Exibe dicas ao passar o mouse sobre widgets.

Métodos:

- `__init__(widget, text)`: Inicializa tooltip
- `show_tooltip()`: Exibe janela com texto
- `hide_tooltip()`: Remove janela

Eventos:

- `<Enter>`: Mouse entra no widget
- `<Leave>`: Mouse sai do widget

8 Tratamento de Erros

8.1 Tipos de Erros Tratados

8.1.1 PermissionError

Causa: Arquivo aberto em outro programa (ex: Excel).

Tratamento:

```
1 try:
2     # Operação com arquivo
3 except PermissionError as e:
4     if e.errno == 13:
5         messagebox.showerror(
6             "Erro de Permissão",
7             "Feche o arquivo e tente novamente"
8         )
```

8.1.2 KeyError

Causa: Coluna esperada não encontrada.

Tratamento:

```
1 try:
2     # Acesso a coluna
3 except KeyError as e:
4     coluna = str(e).strip("'")
5     messagebox.showerror(
6         "Erro de Cabeçalho",
7         f"Coluna '{coluna}' não encontrada"
8     )
```

8.1.3 subprocess.CalledProcessError

Causa: Erro na execução de script externo.

Códigos de Retorno:

- 1: Erro nos arquivos de entrada
- 2: Erro de permissão
- 4: Erro na execução do script

9 Interface Gráfica

9.1 Janela Principal

Botões:

1. Construir Planilha com os dados das Aulas
2. Construir Planilha com os dados do JúpiterWeb

3. Construir Base de Dados do Modelo
4. Construir Base de Dados de Pior Caso
5. Verificar Dados e Executar Modelo
6. Gerar Relatórios
7. Preencher planilhas com Dados da Solução
8. Sair

9.2 Menu de Relatórios

Opções:

1. Relatório de Espaços Livres
2. Visualização Completa da Solução
3. Visualização por Curso
4. Visualização por Departamento
5. Gerar Planilhas para a Intranet

10 Fluxo de Trabalho Típico

10.1 Preparação de Dados

1. Coletar planilhas dos departamentos (SME, SMA, SCC, SSC, Outros)
2. Marcar disciplinas a serem alocadas (coluna "Deve ser alocada no ICMC?")
3. Obter dados do JúpiterWeb
4. Obter dados de ingressantes
5. Obter planilha de espelhos

10.2 Construção das Bases

1. Base das Aulas:

- Menu → "Construir Planilha com os dados das Aulas"
- Selecionar arquivo de cada departamento
- Selecionar planilha de salas
- Informar ano dos dados
- Nomear arquivo de saída

2. Base do JúpiterWeb:

- Menu → "Construir Planilha com os dados do JúpiterWeb"
- Selecionar arquivo de cada departamento
- Adicionar arquivos de outros institutos
- Nomear arquivo de saída

3. Base do Modelo:

- Menu → "Construir Base de Dados do Modelo"
- Selecionar Base das Aulas
- Selecionar Base do JúpiterWeb
- Selecionar planilha de ingressantes
- Selecionar planilha de espelhos
- Nomear arquivo de saída

10.3 Execução do Modelo

1. Menu → "Verificar Dados e Executar Modelo"
2. Selecionar Base de Dados do Modelo
3. **Opcional:** Executar "Selecionar Verificação de Dados"
4. Clicar em "Selecionar Modelo"
5. Escolher entre:
 - Executar com Recomendados
 - Executar com Personalizados (configurar parâmetros)
6. Aguardar execução e verificar saída

10.4 Geração de Relatórios

1. Menu → "Gerar Relatórios"
2. Escolher tipo de relatório
3. Selecionar arquivo "Dados da Solução do Modelo.xlsx"
4. Para espaços livres: também selecionar planilha de salas
5. Nomear arquivos de saída
6. Verificar pasta "Saídas da Interface/Planilhas de Dados"

10.5 Preenchimento de Planilhas

1. Menu → "Preencher planilhas com Dados da Solução"
2. Escolher modo (Completo ou Parcial)
3. Selecionar "Dados da Solução do Modelo.xlsx"
4. **Modo Completo:**
 - Adicionar arquivos de elenco a preencher
 - Opcionalmente: selecionar Base de Dados
5. **Modo Parcial:**
 - Selecionar Base de Dados
 - Marcar aulas a fixar
6. Executar preenchimento
7. Verificar arquivos "...Preenchido.xlsx"

11 Boas Práticas

11.1 Preparação de Arquivos

- Sempre usar formato .xlsx para arquivos Excel
- Não incluir linhas ou colunas vazias entre dados
- Marcar corretamente coluna "Deve ser alocada no ICMC?"
- Padronizar horários: "DIA - HH:MM/HH:MM"
- Fechar arquivos Excel antes de processamento

11.2 Nomenclatura

- Use nomes descritivos para bases de dados
- Inclua ano nos nomes: "Base_Dados_2024.xlsx"
- Evite caracteres especiais
- Mantenha consistência entre arquivos relacionados

11.3 Organização

- Mantenha backups de bases anteriores
- Documente parâmetros usados em cada execução
- Verifique logs de execução em caso de erro
- Compare resultados de diferentes configurações

12 Solução de Problemas

12.1 Problemas Comuns

12.1.1 Erro: Turmas não identificadas

Causa: Células vazias na coluna "Turma".

Solução: Preencher coluna "Turma" para todas as disciplinas marcadas.

12.1.2 Erro: Horário não padronizado

Causa: Formato incorreto na coluna de horário.

Solução: Usar formato "DIA - HH:MM/HH:MM" sem quebras de linha.

12.1.3 Erro: Arquivo está aberto

Causa: Arquivo de saída está aberto no Excel.

Solução: Fechar arquivo e executar novamente.

12.1.4 Erro: Coluna não encontrada

Causa: Cabeçalho incorreto ou ausente.

Solução: Verificar nomes das colunas no arquivo de entrada.

12.2 Logs e Depuração

- Janelas de saída de script mostram progresso em tempo real
- Mensagens de erro incluem linha e coluna problemáticas
- Verificar pasta de saídas para arquivos intermediários
- Revisar base de dados gerada antes de executar modelo

13 Apêndices

13.1 Apêndice A: Formato de Planilhas de Entrada

13.1.1 Planilha de Departamento

Colunas Obrigatórias:

- Disciplina (código)
- Turma
- Horário 1
- Horário 2
- Sala (a definir)
- Utilizará laboratório? (sim ou não)

- Deve ser alocada no ICMC?

Colunas Opcionais:

- Horário 3
- Horário 4
- Observações

13.1.2 Planilha de Salas**Colunas:**

- Sala: Identificador da sala (ex: "3-009")
- Capacidade: Número de lugares
- Tipo: Sala comum, laboratório, etc.

13.2 Apêndice B: Saídas do Sistema**13.2.1 Dados da Solução do Modelo****Colunas:**

- Disciplina: Código-Turma
- Horário: Formato intranet
- Sala: Sala alocada
- Inscritos: Número de alunos
- Cursos: Lista de cursos atendidos
- Nomes: Nome completo da disciplina
- NUSP: Número USP do docente

13.2.2 Visualização Completa**Formato:**

- Arquivo Excel (.xlsx)
- Uma tabela por sala
- Grade horária: 07:00 às 23:30
- Células mescladas para aulas
- Código de cores: verde para ocupado

13.2.3 Planilha de Espaços Livres

Colunas:

- Sala: Identificador da sala
- Dia da semana: Segunda a Sábado
- Horário vago: Intervalos disponíveis

13.2.4 Planilhas da Intranet

Formato: CSV com separador ';' e encoding 'latin-1'

Arquivos gerados:

- Planilha da Intranet Completa.csv
- Planilha da Intranet - [SALA].csv (uma por sala)

13.3 Apêndice C: Estrutura do Código

13.3.1 Imports

```
1 import tkinter as tk
2 from tkinter import ttk, simpledialog, filedialog, messagebox,
   scrolledtext
3 import pandas as pd
4 import subprocess
5 import os
6 import openpyxl
7 from openpyxl import Workbook, load_workbook
8 from openpyxl.styles import PatternFill, Alignment, Border, Side
9 from datetime import datetime, timedelta
10 import sys
11 import re
```

13.3.2 Funções por Categoria

Padronização e Validação:

- padroniza_dataframe()
- ler_df()
- padronizar_horario_intranet()
- extrair_horarios()

Construção de Bases:

- planilha_dep()
- concat_df()

- `base_dados()`
- `base_pior_caso()`

Execução:

- `execute()`
- `Novo_edit_config()`
- `roda_script()`

Visualizações:

- `visualizacao_completa()`
- `visualizacao_curso()`
- `visualizacao_dep()`
- `funcao_visualizacao()`
- `preencher_planilha()`
- `ajusta_largura()`

Análises:

- `analise_vazios()`
- `criar_visualizacao_de_vazias()`
- `get_cell_color()`

Preenchimento:

- `preencher_planilha_dados()`
- `escolhas_preenchimento()`
- `preenchimento()`

Relatórios:

- `gerar_relatorios()`
- `menu_relatorios()`
- `planilhas_sti()`

Interface:

- `Tooltip` (classe)
- `atualizar_estado()` (múltiplas)
- `salvar_valores()` (múltiplas)

13.4 Apêndice D: Algoritmos Principais

13.4.1 Detecção de Intervalos Livres

```
1 ALGORITMO: Detectar_Intervalos_Livres
2 ENTRADA: planilha com alocações, horários
3 SAÍDA: lista de intervalos disponíveis
4
5 PARA CADA sala em salas_utilizadas:
6     PARA CADA dia em dias_da_semana:
7         inicio_livre = NULL
8         PARA CADA horário em lista_horários:
9             célula = obter_célula(sala, dia, horário)
10
11             SE célula está vazia E não é mesclada:
12                 SE inicio_livre é NULL:
13                     inicio_livre = horário
14             SENÃO:
15                 SE inicio_livre não é NULL:
16                     intervalo = [inicio_livre, horário]
17                     ADICIONAR intervalo à lista
18                     inicio_livre = NULL
19
20             // Verificar último intervalo
21             SE inicio_livre não é NULL:
22                 intervalo = [inicio_livre, último_horário]
23                 ADICIONAR intervalo à lista
24
25 PARA CADA sala em todas_salas:
26     SE sala não está em salas_utilizadas:
27         PARA CADA dia em dias_da_semana:
28             ADICIONAR [07:00, 23:30] à lista
29
30 RETORNAR lista de intervalos
```

13.4.2 Padronização de Horários

```
1 ALGORITMO: Padronizar_Horário
2 ENTRADA: horário no formato "DIA - HH:MM/HH:MM"
3 SAÍDA: horário padronizado
4
5 // Remover espaços e converter para minúsculas
6 horário = limpar(horário)
7
8 // Extrair componentes
9 dia, inicio, fim = extrair_partes(horário)
10
11 // Converter para objetos de data/hora
12 inicio_dt = string_para_datetime(inicio)
13 fim_dt = string_para_datetime(fim)
14
```

```

15 // Padronizar início
16 SE inicio_dt.hora < 18:
17     inicio_dt.minutos = 0
18
19 // Padronizar fim
20 SE fim_dt.hora <= 18:
21     SE fim_dt.minutos < 30:
22         fim_dt = subtrair_1_hora(fim_dt)
23         fim_dt.minutos = 30
24     SENÃO SE fim_dt.minutos > 30:
25         fim_dt.minutos = 30
26 SENÃO:
27     SE 0 < fim_dt.minutos < 30:
28         fim_dt.minutos = 0
29     SENÃO SE fim_dt.minutos > 30:
30         fim_dt.minutos = 30
31     SENÃO:
32         fim_dt = subtrair_1_hora(fim_dt)
33         fim_dt.minutos = 30
34
35 RETORNAR formatar(dia, inicio_dt, fim_dt)

```

13.4.3 Preenchimento de Planilhas

```

1 ALGORITMO: Preencher_Planilhas
2 ENTRADA: solução, lista_arquivos, base_dados
3 SAÍDA: arquivos preenchidos
4
5 // Carregar solução
6 solução = ler_excel(arquivo_solução)
7
8 // Processar elencos
9 PARA CADA arquivo_elenco em lista_arquivos:
10     elenco = ler_excel(arquivo_elenco)
11
12     // Encontrar cabeçalho
13     linha_cabeçalho = encontrar_cabeçalho(elenco)
14     elenco = ler_excel_com_cabeçalho(arquivo_elenco, linha_cabeç
15         alho)
16
17     // Abrir como workbook para edição
18     wb = abrir_workbook(arquivo_elenco)
19     ws = wb.planilha_ativa
20
21     PARA CADA linha em elenco:
22         disciplina = criar_código_completo(linha)
23
24         SE disciplina existe em solução:
25             // Filtrar aulas da disciplina
26             aulas = solução[solução.disciplina == disciplina]

```

```

27         // Limpar coluna de salas
28         ws[linha, coluna_sala] = VAZIO
29
30         // Preencher salas por horário
31         PARA i de 1 até 4:
32             horário_atual = linha[f"Horário {i}"]
33
34             PARA CADA aula em aulas:
35                 SE aula.horário == horário_atual:
36                     SE célula já tem valor:
37                         novo_valor = valor_atual + ", " + aula.
38                             sala
39                     SENÃO:
40                         novo_valor = aula.sala
41
42                     ws[linha, coluna_sala] = novo_valor
43
44     SALVAR wb como arquivo_elenco + " Preenchido.xlsx"
45
46 // Processar base de dados (se fornecida)
47 SE base_dados não é NULL:
48     base = ler_excel_multiplas_sheets(base_dados)
49
50     PARA CADA sheet em base.sheets[1:]: // Pular sheet de salas
51         PARA CADA linha em sheet:
52             disciplina = linha.disciplina_código
53
54             SE disciplina existe em solução:
55                 aulas = solução[solução.disciplina == disciplina]
56                 salas = ["0", "0", "0", "0"]
57
58             PARA i de 1 até 4:
59                 horário = linha[f"Horário {i}"]
60
61                 PARA CADA aula em aulas:
62                     SE aula.horário == horário:
63                         salas[i-1] = aula.sala
64
65                 linha.sala = juntar(salas, ", ")
66
67     SALVAR base como base_dados + " Preenchido.xlsx"
68
69 RETORNAR lista de arquivos criados

```

13.5 Apêndice E: Mensagens de Erro e Soluções

13.6 Apêndice F: Configurações Avançadas

13.6.1 Modificação de Intervalos de Horário

Para alterar o intervalo de horários nas visualizações, modificar:

Mensagem de Erro	Solução
"A(s) linha(s) [...] possuem turmas não identificadas"	Preencher coluna "Turma" nas linhas indicadas
"Há um horário de aula não padronizado"	Corrigir formato para "DIA - HH:MM/HH:MM" sem quebras de linha
"A coluna [...] não foi encontrada"	Verificar cabeçalho do arquivo de entrada e garantir que todas as colunas obrigatórias existem
"O arquivo está aberto em algum programa"	Fechar arquivo Excel ou outro programa que o esteja utilizando
"Nenhum arquivo dos outros institutos foi selecionado"	Adicionar arquivos ou confirmar continuação sem eles
"Selecione uma base de dados"	Clicar no botão correspondente e escolher arquivo .xlsx
"Digite um peso de [parâmetro]"	Preencher campo numérico do parâmetro selecionado
"Erro ao executar o script"	Verificar arquivos de entrada e consultar log de saída

Tabela 2: Mensagens de Erro Comuns e Soluções

```

1 # Localizar no código:
2 start_time = datetime.strptime("07:00", "%H:%M")
3 end_time = datetime.strptime("23:30", "%H:%M")
4
5 # Alterar para o intervalo desejado

```

13.6.2 Adição de Novos Departamentos

Para incluir novos departamentos nas visualizações:

```

1 # Na função visualizacao_dep(), localizar:
2 departamentos = ['SME', 'SMA', 'SCC', 'SSC', 'Outras']
3
4 # Adicionar novo departamento à lista:
5 departamentos = ['SME', 'SMA', 'SCC', 'SSC', 'NOVO', 'Outras']

```

13.6.3 Adição de Novos Cursos

Para incluir novos cursos nas visualizações:

```

1 # Na função visualizacao_curso(), localizar:
2 curriculos = ['BMACC', 'BMA', 'LMA', 'MAT-NG',
3               'BECD', 'BCC', 'BSI', 'BCDados']
4
5 # Adicionar novo código de curso à lista

```

13.6.4 Personalização de Cores

Para alterar cores nas visualizações:

```

1 # Localizar definições de PatternFill:
2 green_fill = PatternFill(

```



```
3     start_color="99CC00", # Código hexadecimal da cor
4     end_color="99CC00",
5     fill_type="solid"
6 )
7
8 # Modificar código hexadecimal para nova cor
```

13.7 Apêndice G: Glossário

Alocação: Atribuição de uma disciplina/aula a uma sala específica.

Base de Dados: Arquivo Excel consolidando informações de múltiplas fontes.

Dataframe: Estrutura de dados do Pandas similar a uma tabela.

Elenco: Planilha com disciplinas de um departamento específico.

Espelho: Disciplinas com múltiplas turmas que compartilham vagas.

ICMC: Instituto de Ciências Matemáticas e de Computação.

JúpiterWeb: Sistema de gestão acadêmica da USP.

Merge/Mesclar: Combinar múltiplas células em uma única célula.

NUSP: Número USP (identificador único de pessoas na USP).

Pior Caso: Cenário com maior demanda possível baseado em dados históricos.

Sheet: Planilha individual dentro de um arquivo Excel.

STI/Intranet: Sistema de informação interno do ICMC.

Superlotação: Situação onde número de alunos excede capacidade da sala.

Tooltip: Texto informativo que aparece ao passar mouse sobre elemento.

Turma: Número identificador de uma oferta específica de disciplina.

Workbook: Arquivo Excel completo (pode conter múltiplas sheets).

14 Referências

14.1 Bibliotecas Python

- Pandas: <https://pandas.pydata.org/docs/>
- Openpyxl: <https://openpyxl.readthedocs.io/>
- Tkinter: <https://docs.python.org/3/library/tkinter.html>

14.2 Documentação USP

- JúpiterWeb: Sistema interno da USP
- Portal ICMC: <https://www.icmc.usp.br/>

15 Controle de Versões

15.1 Histórico de Versões

Versão	Data	Alterações
1.0	-	Versão inicial do sistema Interface gráfica completa Todas funcionalidades implementadas

Tabela 3: Histórico de Versões

15.2 Desenvolvedor

- Código original desenvolvido para o ICMC-USP
- Adaptado de notebooks Colab
- Integração com modelo de otimização

16 Considerações Finais

16.1 Limitações Conhecidas

- Sistema depende de formato específico de planilhas de entrada
- Alterações em estrutura de dados requerem modificação do código
- Laboratórios conjuntos (6-303/6-304, 6-305/6-306) têm tratamento especial
- Validações assumem dados consistentes entre fontes

16.2 Desenvolvimentos Futuros

- Interface web para acesso remoto
- Validação mais robusta de dados de entrada
- Integração direta com JúpiterWeb
- Histórico de alocações por disciplina
- Análises estatísticas de utilização
- Sugestões automáticas de melhorias

16.3 Contato e Suporte

Para questões sobre o sistema, entrar em contato com:

- Seção de Graduação do ICMC
- Seção de Apoio Administrativo
- Equipe de desenvolvimento (a definir)

16.4 Licença

Sistema desenvolvido para uso interno do ICMC-USP. Todos os direitos reservados.

*Documento gerado automaticamente
Versão da documentação: 1.0
Data: 13 de abril de 2026*